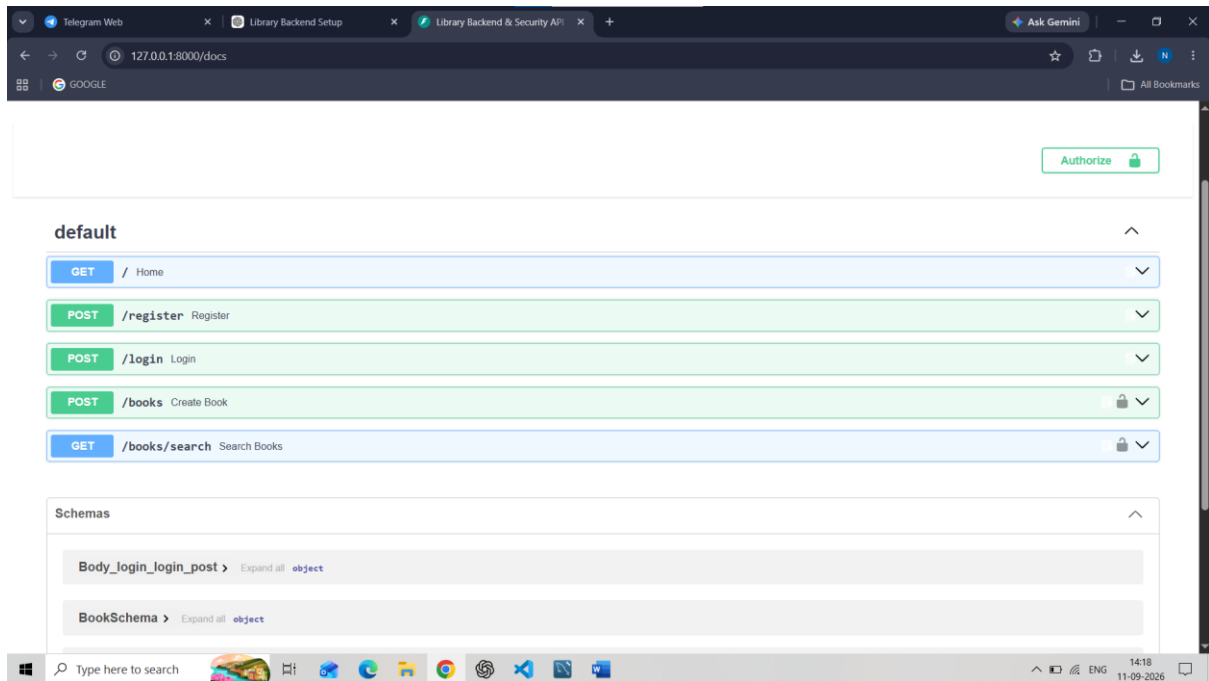


PRACTICAL 6 DBMS



Explorer

...

✓ Open Editors

 *Welcome*

 main.py

✕  database.py

 models.py

 schemas.py

 security.py

 embedding.py

≡ requirments.txt

✓ library_fastapi_project

> __pycache__

 database.py

 embedding.py

 main.py

 models.py

≡ requirments.txt

 schemas.py

 security.py

```
2     AsyncSession,  
3     async_sessionmaker,  
4     create_async_engine  
5 )  
6 from sqlalchemy.orm import DeclarativeBase  
7 from sqlalchemy import URL  
8  
9  
10 DATABASE_URL = URL.create(  
11     "mysql+asyncmy",  
12     username="root",  
13     password="nishanth@123",  
14     host="localhost",  
15     port=3306,  
16     database="nishanth_db"
```

```
20 engine = create_async_engine(  
21     DATABASE_URL,  
22     echo=True  
23 )  
24  
25  
26 AsyncSessionLocal = async_sessionmaker(  
27     engine,  
28     expire_on_commit=False  
29 )  
30  
31  
32 class Base(DeclarativeBase):  
33     pass
```

```
4 VECTOR_DIM = 384
5
6
7 def text_to_vector(text: str) -> list[float]:
8     values = []
9
10    for i in range(VECTOR_DIM):
11        digest = hashlib.sha256(
12            f"{text.lower()}:{i}".encode()
13        ).digest()
14
15        number = int.from_bytes(
16            digest[:4],
17            byteorder="big"
18        )
```

```
20        values.append(
21            (number / 2**32) * 2 - 1
22        )
23
24    magnitude = math.sqrt(
25        sum(x * x for x in values)
26    )
27
28    return [x / magnitude for x in values]
```

```
1 from fastapi import (
2     BackgroundTasks,
3     Depends,
4     FastAPI,
5     HTTPException,
6     Query,
7     status
8 )
9
10 from fastapi.security import OAuth2PasswordRequestForm
11 from sqlalchemy import select
12 from sqlalchemy.ext.asyncio import AsyncSession
13
14 from database import Base, engine, get_db
15 from embedding import text_to_vector
```

```
14 from database import Base, engine, get_db
15 from embedding import text_to_vector
16 from models import Book, Member
17 from schemas import BookSchema, MemberRegister, Token
18 from security import (
19     create_access_token,
20     get_current_member,
21     hash_password,
22     verify_password
23 )
24
25 import math
26
```

```

34 @app.on_event("startup")
35 async def startup():
36     async with engine.begin() as connection:
37         await connection.run_sync(
38             Base.metadata.create_all
39         )
40
41
42 @app.get("/")
43 async def home():
44     return {"message": "Library API is running"}
45
46
47 def log_new_book(book_id: int, title: str):

```

```

47 def log_new_book(book_id: int, title: str):
48     with open(
49         "book_events.txt",
50         "a",
51         encoding="utf-8"
52     ) as file:
53         file.write(
54             f"New Book Added: id={book_id}, title={ti
55         )
56
57
58 def cosine_similarity(
59     a: list[float],
60     b: list[float]

```

```
70     norm_a = math.sqrt(  
71         sum(x * x for x in a)  
72     )  
73  
74     norm_b = math.sqrt(  
75         sum(y * y for y in b)  
76     )  
77  
78     if norm_a == 0 or norm_b == 0:  
79         return 0.0
```

```
86     member: MemberRegister,  
87     db: AsyncSession = Depends(get_db)  
88 ):  
89  
90     result = await db.execute(  
91         select(Member).where(  
92             Member.username == member.username  
93         )  
94     )  
95  
96     if result.scalar_one_or_none():  
97         raise HTTPException(  
98             status_code=400,  
99             detail="Username already exists"
```

```
102     new_member = Member(  
103         username=member.username,  
104         password_hash=hash_password(  
105             member.password  
106         ),  
107         role="member"  
108     )  
109  
110     db.add(new_member)  
111  
112     await db.commit()  
113     await db.refresh(new_member)  
114
```

```
109     }  
110  
111  
112 @app.post(  
113     "/login",  
114     response_model=TokenResponse  
115 )  
116 async def login(  
117     form_data: OAuth2PasswordRequestForm = Depends(),  
118     db: AsyncSession = Depends(get_db)  
119 )
```

```
131     result = await db.execute(  
132         select(Member).where(  
133             Member.username == form_data.username  
134         )  
135     )  
136  
137     member = result.scalar_one_or_none()  
138  
139     if not member or not verify_password(  
140         form_data.password,  
141         member.password_hash  
142     ):
```

```
148         token = create_access_token(  
149             member.username,  
150             member.role  
151         )  
152  
153         return {  
154             "access_token": token,  
155             "token_type": "bearer"  
156         }  
157  
158  
159 @app.post(  

```

```
1 from datetime import date
2 from decimal import Decimal
3
4 from sqlalchemy import Date, Integer, Numeric, String
5 from sqlalchemy.orm import Mapped, mapped_column
6
7 from database import Base
8
9
10 class Member(Base):
11     __tablename__ = "members"
12
13     id: Mapped[int] = mapped_column(
14         Integer,
15         primary_key=True
```

```
20         unique=True,
21         nullable=False
22     )
23
24     password_hash: Mapped[str] = mapped_column(
25         String(255),
26         nullable=False
27     )
28
29     role: Mapped[str] = mapped_column(
30         String(30),
31         default="member",
32         nullable=False
```

```

36 class Book(Base):
37     __tablename__ = "books"
38
39     id: Mapped[int] = mapped_column(
40         Integer,
41         primary_key=True
42     )
43
44     title: Mapped[str] = mapped_column(
45         String(250),
46         nullable=False
47     )

```

blems Output Debug Console Terminal Ports python

```

schemas.py > TokenResponse
7 class BookSchema(BaseModel):
14     ,
15
16     price: Decimal = Field(gt=0)
17
18     published_date: date
19
20     description: str | None = None
21
22     @field_validator("published_date")
23     @classmethod
24     def validate_published_date(cls, value: date):
25         if value > date.today():
26             raise ValueError(
27                 "published_date cannot be in the future"
28             )
29         return value
30
31
32 class MemberRegister(BaseModel):
33     username: str
34     password: str = Field(min_length=6)
35
36
37 class TokenResponse(BaseModel):

```

```
1 from datetime import datetime, timedelta, timezone
2
3 from fastapi import Depends, HTTPException
4 from fastapi.security import OAuth2PasswordBearer
5 from jose import JWTError, jwt
6 from passlib.context import CryptContext
7 from sqlalchemy import select
8 from sqlalchemy.ext.asyncio import AsyncSession
9
10 from database import get_db
11 from models import Member
12
13
14 SECRET_KEY = "CHANGE_THIS_SECRET"
15 ALGORITHM = "HS256"
16 ACCESS_TOKEN_EXPIRE_MINUTES = 60
17
18
19 pwd_context = CryptContext(
20     schemes=["bcrypt"],
21     deprecated="auto"
22 )
23
24 oauth2_scheme = OAuth2PasswordBearer(
25     tokenUrl="/login"
```

```

28
29 def hash_password(password: str):
30     return pwd_context.hash(password)
31
32
33 def verify_password(plain_password, hashed_password):
34     return pwd_context.verify(
35         plain_password,
36         hashed_password
37     )
38
39
40 def create_access_token(username, role):
41     expire = datetime.now(timezone.utc) + timedelta(
42         minutes=ACCESS_TOKEN_EXPIRE_MINUTES
43     )
44
45     payload = {
46         "sub": username,
47         "role": role,
48         "exp": expire
49     },

```

```

69         payload = jwt.decode(
70             token,
71             SECRET_KEY,
72             algorithms=[ALGORITHM]
73         )
74
75         username = payload.get("sub")
76
77         if not username:
78             raise credentials_exception
79
80     except JWTError:
81         raise credentials_exception
82
83     result = await db.execute(
84         select(Member).where(
85             Member.username == username
86         )
87     )
88
89     member = result.scalar_one_or_none()
90
91     if not member:
92         raise credentials_exception

```

GET / Home

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/

Server response

Code

Details

200

Response body

```
{
  "message": "Library API is running"
}
```

Download

POST /register Register

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "username": "FILPH",
  "password": "@123"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/register' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "username": "FILPH",
    "password": "@123"
  }'
```

200 Successful Response No

Media type

application/json

Controls Accept header

Example Value | Schema

```
"string"
```

422 Validation Error No

POST /login Login

Parameters

No parameters

Request body required

application/x-www-form-urlencoded

grant_type

string | (string | null)

password

pattern: "password\$"

☐ Send empty value

username required

string

libraryuser3

password required

string(\$password)

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=libraryuser3&password=test1234&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8000/login

Server response

Code

Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IWRXVz9.eyJzdWIiOiJ3YWVjYXN0XmIjMkIjYybzI1IjoibWVtYmVYIiwiaXNzIjw5RTIyNDI2fQ.8u56LZ2X0n5V6iYTEGe3RAM1-qm8pdJwzx8QeT2eeg",
  "token_type": "bearer"
}
```

Download

Response headers

```
content-length: 196
content-type: application/json
date: Fri, 11 Sep 2026 09:26:46 GMT
server: uvicorn
```

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header:

POST

/books

Create Book

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "title": "Python for Beginners",
  "author": "John Smith",
  "isbn": "9781234567890",
  "price": 499,
  "published_date": "2024-06-10",
  "description": "An introduction to Python programming."
}
```

Execute

Clear

Responses

Request URL

http://127.0.0.1:8000/books

Server response

Code	Details
201	Response body <pre>{ "message": "Book added successfully", "book_id": 1, "added_by": "libraryuser3"}</pre> Response headers <pre>content-length: 75 content-type: application/json date: Fri, 11 Sep 2026 09:34:20 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No link
422	Validation Error Media type application/json Example Value Schema	No link

GET

/books/search

Search Books



Parameters

Cancel

Name	Description
q * required	
string	
(query)	
	python
	minLength: 1
limit	
integer	
(query)	
	5
	maximum: 20
	minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8080/books/search?q=python&limit=5' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IWRkcCI6ImlkCjYyZDh1IjoibWVYWyIiwiaXNpdjoxNzgzNTIyODI0I0FQ.GXvtKd5k3AVvUkSLMK3QyBg_SU2hgQx2VST2X_9q1cw'
```

Request URL

http://127.0.0.1:8080/books/search?q=python&limit=5

Server response

Code

Details

200

Response body

```
{
  "query": "python",
  "results": [
    {
      "id": 1,
      "title": "Python for Beginners",
      "author": "John Smith",
      "isbn": "9781234567890",
      "price": 450,
      "published_date": "2024-06-10",
      "description": "An introduction to Python programming.",
      "similarity": 0.03789975769122376
    }
  ]
}
```



Download

Response headers

Code

Details

200

Response body

```
{
  "query": "python",
  "results": [
    {
      "id": 1,
      "title": "Python for Beginners",
      "author": "John Smith",
      "isbn": "9781234567890",
      "price": 450,
      "published_date": "2024-06-10",
      "description": "An introduction to Python programming.",
      "similarity": 0.03789975769122376
    }
  ]
}
```



Download

Response headers

```
content-length: 247
content-type: application/json
date: Fri, 11 Sep 2026 09:38:18 GMT
server: uvicorn
```